

PYTHON WEEKLY ASSIGNMENT

BUILDER QUEST

5 guided missions using loops, data structures and functions

THE GOAL

This week you learned how to organize data and turn logic into reusable functions.

Now put those skills together and build small programs that feel like real mini-tools.

Your skill stack

LISTS + DICTIONARIES + TUPLES
LOOPS + IF/ELSE + FUNCTIONS + RETURN

How to use this assignment

1. Read the story.
2. Follow the guided steps.
3. Try the code yourself.
4. Check the expected output.
5. Try the optional level-up only after the main task works.

The five missions

01	Movie Night Picker
02	Student Score Coach
03	AI Result Reporter
04	Mini Expense Analyzer
05	Smart Study Dashboard

Challenge rule: The main task is beginner-friendly. The level-up gives an extra challenge so students with some experience do not feel bored.

Turn a list of movie titles into a tiny recommendation tool.

The story

You and your friends have a list of movies, but nobody wants to scroll through the whole list. Build a small tool that searches the list for a movie the user wants.

Starting data

```
movies = ["Inception", "Interstellar", "Coco", "The Batman", "Dune"]
```

Your mission

- 1 Create a function called `find_movie(movies, wanted)`.
- 2 Loop through the list.
- 3 If the movie matches the user's search, return a helpful message.
- 4 If nothing matches, return a 'not found' message.
- 5 Ask the user which movie they want to find and print the returned result.

Example run

```
Enter movie: Coco
```

```
Found: Coco
```

Try another

```
Enter movie: Avatar
```

```
Movie not found.
```

Hint: Start with a function that accepts the list and the search text. Use a for loop and an if statement. Use return when you know the answer.

LEVEL UP

Make the search ignore capital letters so 'coco' and 'Coco' both work.

The story

You have student records stored as dictionaries inside a list. Your program should find strong performers and give them a simple message.

```
students = [  
    {"name": "Ali", "score": 82},  
    {"name": "Sara", "score": 91},  
    {"name": "Hamza", "score": 67},  
    {"name": "Ayesha", "score": 88}  
]
```

Your mission

- 1 Create `is_high_score(score)` that returns `True` when score is 80 or above.
- 2 Create `show_student(student)` that prints the name and score.
- 3 Loop through the student list.
- 4 Use `is_high_score()` to decide who should be shown.

Expected output

```
Ali - 82  
Sara - 91  
Ayesha - 88
```

Think: The list stores many students. Each student dictionary stores named information. Functions keep the logic reusable.

LEVEL UP

Create `get_grade(score)` that returns A, B, C or Needs Improvement, then include the grade in the output.

Pretend your AI model just returned a prediction.

The story

An AI image classifier returns structured information. Your job is to turn that data into a clean human-readable report.

```
result = {  
    "label": "cat",  
    "confidence": 0.94,  
    "class_id": 3  
}
```

Your mission

- 1 Create a function `show_result(result)`.
- 2 Read the dictionary values using their keys.
- 3 Print the label, confidence and class ID.
- 4 Add a decision: if confidence is 0.80 or above, print 'Confident prediction'; otherwise print 'Low confidence'.

Expected output

```
Label: cat  
Confidence: 0.94  
Class ID: 3  
Confident prediction
```

Why this matters: AI applications constantly move structured information between components. Dictionaries are a simple way to represent that kind of named result.

LEVEL UP

Change the confidence to 0.62 and make sure your program changes the message automatically.

Mini Expense Analyzer

Use functions + lists to turn simple data into useful numbers.

The story

You tracked a few daily expenses. Build a small analyzer that calculates a total and tells you which expenses are above a chosen limit.

```
expenses = [250, 400, 120, 600, 300]
```

Your mission

- 1 Create `get_total(expenses)` that uses a loop and returns the total.
- 2 Create `show_large_expenses(expenses, limit)` that prints values above the limit.
- 3 Use the functions with the given list.

Expected output

```
Total expense: 1670
```

```
Expenses above 300:  
400  
600
```

Hint: For the total, start with `total = 0` and keep adding each value inside the loop.

LEVEL UP

Ask the user for the limit instead of hard-coding 300.

The story

You are building a tiny study dashboard. It receives structured course data and produces a simple summary.

```
courses = [  
    {"name": "Python", "score": 88},  
    {"name": "Git", "score": 92},  
    {"name": "Math", "score": 74},  
    {"name": "SQL", "score": 81}  
]
```

Your mission

- 1 Create `is_strong(score)` that returns True for scores 80+.
- 2 Create `show_course(course)` that prints course name and score.
- 3 Create `average_score(courses)` that uses a loop and returns the average.
- 4 Loop through the courses and show only strong courses.
- 5 Print the average score at the end.

Expected output

```
Strong courses:  
Python - 88  
Git - 92  
SQL - 81  
  
Average score: 83.75
```

What you are combining:

List + dictionary + loop + if + function + parameter + return

LEVEL UP

Add a function `get_status(score)` that returns 'Excellent', 'Good' or 'Needs Improvement'.

Final goal: Make the dashboard easy to read and easy to modify.

QUEST COMPLETE

YOUR SUBMISSION CHECKPOINT

Before you submit:

- ☐ All 5 programs run without errors
- ☐ I tested different inputs
- ☐ I used functions instead of repeating logic
- ☐ I used lists/dictionaries where they make sense
- ☐ I can explain my own code
- ☐ I attempted at least one Level Up

What this assignment is really testing

Can you look at a small problem and decide...

What data structure fits?
What logic should become a function?
Where do I need a loop?
Where do I need a condition?
What result should I return?

Remember: You are not trying to write the most complicated code. You are trying to build a clean solution that you understand.

THIS WEEK'S SKILL STACK

LOOPS -> DATA STRUCTURES -> FUNCTIONS
↓
MINI AI / DATA SYSTEMS

The long-term goal is to use Python as an engineering tool for Data Science, Machine Learning and AI systems.